

DyCGraph

1 Introduction

2 Background and Motivation

2.1 Streaming Graph Processing

A graph represents entities as vertices and their relationships as edges. Graph algorithms derive properties from these relationships: breadth-first search (BFS) computes hop distances from a source, single-source shortest paths (SSSP) computes minimum path weights, and connected components (CC) identifies connected groups in an undirected graph. Many algorithms maintain a state for each vertex and iteratively exchange information along edges. Vertices requiring processing form an active set, or *frontier*: their neighbors are examined, states are updated, and further work is generated until the algorithm terminates.

In streaming applications, a streaming graph evolves through continuous edge insertions and deletions. We group these changes into batches over a fixed vertex set. Let $G_t = (V, E_t)$ be the current graph, ΔG_t an update batch, and $\mathcal{A}$ a graph algorithm. Applying the batch produces the updated graph G_{t+1} , for which the result must be maintained:

$$G_{t+1} = \text{Apply}(G_t, \Delta G_t), \quad R_{t+1} = \mathcal{A}(G_{t+1}).$$

Streaming graph updates often modify only a portion of the graph, and their effects on algorithm results can also be confined to a subset of vertices. We refer to this property of graph modifications and their computational effects as *regionality*. For SSSP, inserting an edge may introduce a shorter path, whereas deleting an edge supporting a shortest path may invalidate distances several hops away.

2.2 CPU Graph Processing and Incremental Systems

Early research on graph processing primarily focused on CPU-based systems. To accelerate graph traversal and iterative computation, [Ligra] provides parallel vertex and edge mapping primitives and adapts traversal strategies to the density of the active vertex set. Another line of work optimizes graph storage and access under dynamic updates. [STINGER] organizes adjacency lists as linked edge blocks and supports parallel batch updates. [Terrace] uses degree-dependent storage, combining in-place arrays, a shared packed memory array, and per-vertex B-trees to balance update and adjacency-scan efficiency. For concurrent updates and analytics, [GraphOne] combines an edge log with adjacency storage and uses dual versioning to decouple ingestion from computation, while [LiveGraph] employs a multiversion Transactional Edge Log to support transactional updates with sequential adjacency scans.

Several studies have explored how to reuse previously computed results when a streaming graph changes. Recomputing from scratch on G_{t+1} repeats work for vertices whose results remain unchanged. Incremental computation exploits the regionality of graph updates by retaining valid results and recomputing the affected region. The following systems track dependencies between vertex values to

determine which results are affected and propagate corrections through these dependencies.

[KickStarter] exploits this principle for a class of monotonic graph algorithms: it tracks value dependencies and trims intermediate values invalidated by edge deletions, enabling computation to resume from a valid approximation. To support algorithms requiring Bulk Synchronous Parallel (BSP) semantics, [GraphBolt] extends dependency tracking across iterations and incrementally refines intermediate values while preserving synchronized execution semantics. However, this refinement can continue propagating changes even when vertex computations become sparse. [DZiG] addresses this limitation through a recursive formulation of incremental computation that identifies and prunes unnecessary updates while retaining BSP semantics.

2.3 Out-of-Memory GPU Graph Processing

GPUs offer massive parallelism and high memory bandwidth for processing vertices and edges. Dynamic GPU structures also support changing adjacency lists: [Hornet] allocates adjacency storage in blocks with power-of-two capacities, while [faimGraph] uses linked pages and GPU-managed memory recycling to support both vertex and edge updates. These designs improve updates within device memory, but large real-world graphs frequently exceed the memory capacity of a single GPU [EMOGI].

Out-of-memory (OOM) GPU processing addresses this capacity limit by retaining graph data in host memory and making the required portions accessible to the GPU. In a common arrangement, vertex states and adjacency indices reside on the GPU, while the larger edge arrays reside in host memory. This heterogeneous memory organization need not imply that both processors execute graph algorithms. Accessing host data across the interconnect is much slower than accessing GPU memory, making data movement a central concern.

For static graphs, [Subway] constructs subgraphs containing only active edges and processes them asynchronously to reduce repeated generation and transfer. [EMOGI] instead uses zero-copy access, allowing GPU threads to read host-resident edges directly through coalesced, aligned cache-line requests. Zero-copy avoids explicit bulk copying, but data still crosses the interconnect. [HyT-Graph] uses cost estimates to choose between explicit transfers and zero-copy access for graph partitions, and combines these choices with asynchronous scheduling. [CGgraph] retains a reusable sub-graph in GPU memory across iterations and allocates tasks on demand between the CPU and GPU to support cooperative computation.

[GrapiN] brings incremental computation to OOM streaming graphs. It maintains dynamic topology in host memory and executes dependency-based incremental analysis on the GPU, separating result and dependency updates into GPU-native atomic operations. It also uses zero-copy to provide fine-grained access to graph data that are not resident in GPU memory. However, its

topology maintenance and incremental execution do not fully exploit the regionality of streaming graph updates and their effects on computation.

2.4 Research Challenges

Exploiting regionality in topology maintenance. A batch may modify only a small set of adjacency lists, yet maintaining the graph can require changes to data outside this region. Grapin’s packed memory array uses segments and reserved gaps to limit the scope of rebalancing [Grapin]. When nearby space is insufficient, however, an insertion can still move neighboring adjacency lists, redistribute data across a larger range, or require storage expansion. The cost extends beyond moving data on the CPU: changes in adjacency locations must also be reflected in the GPU indices before traversal can resume, adding synchronization and transfer overhead to the original update.

Exploiting regionality in result changes. The vertices affected by an update are not limited to the endpoints of the modified edges. For example, deleting an edge on a shortest path can invalidate distances further along that path. Finding an alternative path may also require reading neighbors whose distances have not changed. Processing only the update endpoints can therefore miss necessary corrections, whereas restarting across the graph repeats unaffected work. Even within the affected region, a vertex may be processed several times as shorter paths are discovered. The challenge is to identify and repair all affected results while avoiding unnecessary processing and repeated propagation.

The next chapter presents our system design for addressing these challenges in graph maintenance and incremental computation.

3 System Overview

We present an out-of-memory streaming graph system that exploits regionality in both topology changes and their computational effects. The system processes batches of edge insertions and deletions over a fixed vertex set, retaining graph topology in host memory and performing incremental computation on the GPU. Our design combines **update-friendly heterogeneous graph organization** (Section 4) with **impact-driven incremental graph computation** (Section 5). We use SSSP to illustrate the design.

3.1 Design Rationale

A batch of streaming graph updates often modifies only a small part of the topology, and its effects on algorithm results may also remain within a limited region. This regionality provides opportunities to reduce both graph maintenance and recomputation. The two regions need not coincide: deleting one edge can affect results several hops away, and recovery may require reading unchanged vertices. Our design uses the scope of topology changes to guide data maintenance and algorithm dependencies to guide incremental computation.

Update-friendly heterogeneous graph organization. Local topology changes should remain local as they pass from host storage to GPU views. We organize outgoing adjacency lists in independently allocated blocks, so growing one list does not relocate unrelated lists. A unified change view records the updates that actually take effect and supplies consistent changes to the forward

topology, reverse index, and GPU publication path. This organization allows the GPU adjacency descriptors to be updated only for sources whose adjacency lists have changed (Section 4).

Impact-driven incremental graph computation. Prior work on incremental graph computation initializes worklists over all vertices, and its partition-based scheduling may scan edges unrelated to the affected computation. We use a compact queue of invalidated vertices to drive incoming-edge preparation and GPU recovery, avoiding full-graph state gathering and partition worklist reconstruction (Section 5.1). Recovery completes before insertions, leaving no pending distance improvements on existing edges; only added edges therefore need to seed insertion processing. Successful improvements directly enqueue vertices for adjacency expansion, while GPU-resident queues advance propagation to convergence without host coordination between waves (Section 5.2). Together, these mechanisms replace all-vertex startup and partition-wide scans with work on affected vertices and their dependencies. For long propagations, optional distance ordering reduces repeated edge scans; for large batches, shared plans and change records reduce repeated preparation (Section 5.3).

3.2 System Workflow

Figure 1 shows how graph organization (Section 4) supports incremental computation (Section 5). The CPU maintains topology, while the GPU retains vertex states and executes recovery and propagation. Each batch proceeds as follows.

1. Prepare updates. The CPU groups the batch once by source, retaining separate deletion and insertion views. Our unified change view supplies the adjacency store, reverse index, and publication path with the same effective changes, avoiding independent interpretations of update requests.

2. Recover from deletions. GPU dependency invalidation directly produces a compact affected-vertex queue. After applying deletions, the CPU prepares incoming edges only for queued vertices, and the GPU recovers their distances to convergence using unchanged boundary values. The same queue bounds preparation and recovery, avoiding full-graph state gathering and partition worklist reconstruction; completing recovery makes added-edge startup sufficient in step 4.

3. Insert and publish. The CPU applies insertions and combines the sources changed in both phases. Our source-isolated layout keeps unrelated adjacency addresses stable, allowing the GPU to receive only changed-source descriptor patches instead of reloading all descriptors. Publication completes before dependent GPU reads.

4. Propagate improvements. Successful relaxations of added edges directly populate the GPU work queue. Each queued vertex expands its own outgoing adjacency, avoiding scans of unrelated vertices and edges in its graph partition. The GPU forms subsequent frontiers and detects convergence without host intervention between waves. Per-wave deduplication prevents concurrent processing of the same vertex.

4 Update-Friendly Heterogeneous Graph Organization

Topological regionality offers an opportunity to limit adjacency movement and GPU index maintenance to the sources whose outgoing adjacency changes in a batch. However, insertion operations in the PMA structure may trigger rebalancing, causing the adjacency list to be migrated to a region other than that of the source vertex, thereby increasing the volume of data movement and the overhead of index maintenance at other locations. We organize maintenance around independently allocated source blocks. Section 4.1 explains how this layout prevents updates from relocating unrelated lists. Section 4.2 coordinates updates to outgoing and incoming adjacency through a common batch plan. Section 4.3 uses the resulting changed-source set to update only the required GPU adjacency indices while preserving consistent graph access.

4.1 Source-Isolated Graph Layout

The graph resides in host memory. For each source, the GPU maintains an adjacency descriptor, an index record that identifies where its neighbor list is stored and how many neighbors it contains. Although graph updates are regional, modifying a few vertices can relocate the adjacency lists of many unchanged vertices, requiring corresponding updates to their GPU indices and thereby amplifying the data update overhead. To confine this maintenance to updated sources, we allocate and grow each source’s adjacency independently.

Each source u stores its neighbors in an independently allocated, contiguous block of mapped host memory, with spare capacity for insertions. For insertions, the CPU appends new neighbors to u ’s existing block if its capacity is sufficient. Otherwise, it allocates a larger block, copies u ’s existing adjacency into it, and appends the new neighbors. Expansion uses a separately allocated block, leaving the locations of other adjacency lists unchanged. Figure 2(a) contrasts this with a shared PMA: even a nearby shift before rebalancing can change the start index of an unchanged source v , whereas expanding u ’s independent block leaves v ’s block and descriptor valid. For deletions, the CPU removes the specified edges that exist in the graph and compacts the remaining neighbors within the existing block. This preserves the contiguous neighbor access of the original layout while restricting relocation to the updated source.

Independent block allocation eliminates cross-source redistribution, preserving the adjacency locations and descriptors of unchanged sources. This confines relocation costs to updated lists and enables sparse GPU publication (Section 4.3). Expansion and deletion may still copy or scan an entire updated list, while spare capacity and delayed block reclamation incur memory overhead.

4.2 Unified Change View

A topology view is a representation of graph in a particular data structure or on a particular device, organized for the access it supports. Our system maintains three such views. Outgoing adjacency stores each source’s neighbors in host memory for traversing its outgoing edges. The reverse index stores the sources of incoming edges for each destination, allowing deletion recovery to find the remaining predecessors that may restore its distance. GPU adjacency

descriptors store each source’s neighbor-list location and degree, allowing GPU computation to locate and read its outgoing neighbors. Each topology update must be reflected in all three views. We coordinate their maintenance through a unified change view, consisting of effective edge-change records and changed-source lists produced during outgoing-adjacency preparation for each update phase.

The CPU groups update requests by source so that adjacency maintenance operates on a group of edges at a time. For each source, it collects all deletion requests and records how many existing edges to each destination should be removed. It then removes these edges together by compacting the neighbor list once, avoiding repeated movement of the same entries when edges are deleted one at a time. It also determines the capacity required for the source’s insertions as a group. If the current block is too small, the CPU allocates a larger block and copies the existing neighbors once before appending the new edges. Thus, grouping reduces repeated movement within an updated adjacency list, complementing the isolation of movement across sources in Section 4.1. Independent source blocks allow the compaction and append operations for different sources to proceed in parallel.

Grouping updates by source also provides the information needed to maintain the other topology views. During preparation, the CPU records each source–destination pair and the number of edges actually removed or added, together with the sources whose adjacency changes. Unsuccessful deletion requests produce no change. The reverse index uses the edge records to update incoming adjacency, while GPU descriptor and cache updates use the source lists to locate the outgoing lists that have changed. These structures therefore reuse the results of adjacency preparation instead of independently determining the effects of the same requests.

For example, suppose source u has neighbors $[a, a, b]$ and receives three requests to delete (u, a) and one to insert (u, c) . Only two copies of (u, a) exist, so deletion compacts the list to $[b]$ and produces the record $(u, a, -2)$. The reverse index uses this record to remove the two incoming edges from u at a . Insertion later produces $[b, c]$ and the record $(u, c, +1)$, which adds an incoming edge from u at c . Both stages identify u as changed. The system then updates its GPU descriptor and, if present, its cached adjacency, as illustrated in Figure 2(b). The third deletion request causes no additional index update.

4.3 Maintaining Reverse and GPU Views

Reverse index maintenance. Maintaining incoming adjacency directly from the input requests would cause other views to repeat checks already performed when processing outgoing adjacency. To avoid this work, the reverse index consumes the actual edge changes, groups them by destination, and combines changes for the same source–destination pair. Each destination stores a sorted base list of incoming sources and a separate list of delta records. A delta record contains a source vertex and the net number of edges added or removed relative to the base list: a positive value indicates additions, and a negative value indicates deletions. New changes are merged into these records, and records with a zero net change are removed. This avoids checking deletion requests again and modifying the base lists for every update.

To retrieve the current incoming neighbors of a destination, the index counts the occurrences of each source in the base list and adds the value in its delta record, if one exists. A source is returned if the resulting count is positive. For example, a base list $[u, u, w]$ and delta records $[(u, -2), (v, +1)]$ yield incoming sources $[v, w]$: both edges from u have been removed, the edge from w remains, and an edge from v has been added. The base list remains unchanged during updates, while the delta records provide the current incoming view at query time.

GPU descriptor and cache maintenance. Reloading all GPU descriptors after a local update would transfer indices for many vertices whose adjacency remains unchanged. Source isolation makes these transfers unnecessary, while the source lists produced during adjacency maintenance identify the descriptors that need updating. Let S^- and S^+ denote the sources changed during deletion and insertion. The system forms $S = S^- \cup S^+$ so that each source is included once, even if it changes in both stages. Sources with only unsuccessful deletion requests are excluded. After insertion, the CPU transfers one record per source in S , containing its final adjacency location, degree, and version, and the GPU writes these records into the corresponding descriptor entries. Sources outside S retain valid descriptors because their adjacency has not moved or changed. Descriptor maintenance therefore follows the sources actually modified in the batch, including those updated in place or restored to their initial neighbors, without reloading the full array.

Each descriptor update occupies 24 bytes, giving a transfer volume of $24|S|$ bytes per batch. Descriptor traffic therefore scales with the number of changed sources rather than the total vertex count. Incoming-neighbor transfers, cached-list copies, and accesses to host adjacency incur separate costs.

The source list also limits the repair of cached adjacency. If a changed source has a valid GPU cache entry, the system copies its current neighbor list into the existing space when it fits. Otherwise, it attempts to place the list in unused space at the end of the cache and updates its cache index. If there is insufficient space, it invalidates the cached copy, allowing computation to read the current host adjacency. Each repair copies the changed source’s entire neighbor list; cached lists of other sources are left in place during this step.

To prevent GPU computation from reading stale indices or cached neighbors, descriptor and cache updates run in one CUDA stream, and subsequent computation waits for them to finish. Replaced host blocks are reused only after earlier GPU readers have completed, preventing access to reclaimed adjacency storage.

Local repair can also avoid subsequent cache reorganization. After computation, the existing hotness-based policy selects the vertices to cache. If all selected vertices have valid cached adjacency and their number matches the previously published cache set, the system skips eviction, compaction, and loading. Otherwise, it performs the normal cache replacement procedure. This prevents a local topology change from unnecessarily triggering the full cache maintenance sequence, while still allowing replacement when the selected vertices change or local repair cannot retain a valid copy.

Together, source isolation, shared change records, and selective GPU updates confine adjacency relocation and cache repair to changed sources, while avoiding repeated processing of the same

updates across topology indices. They reduce both data movement and the additional index maintenance.

5 CPU–GPU Cooperative Incremental Graph Computation

Updates to the streaming graph are localized, yet work list construction and incremental maintenance can still process unaffected vertices. We reduce this overhead through CPU–GPU cooperative incremental computation. Section 5.1 uses affected vertices identified by the GPU to retrieve only the predecessor lists needed for deletion repair from the CPU-maintained reverse index. Section 5.2 publishes changed topology and propagates insertion effects only from vertices whose results improve. Section 5.3 reduces repeated propagation and large-batch preparation costs.

5.1 Dependency Invalidation and Cooperative Repair

We describe the process using SSSP with positive edge weights (BFS uses unit weights). A vertex result is its current distance estimate, denoted by $d(v)$, and a better result corresponds to a smaller estimate in this example. For a batch of effective deletions D_t and insertions I_t , let $G_t = (V, E_t)$ be the previous graph. The graph after deletion is $G_{t+1}^- = (V, E_t \setminus D_t)$, and the final graph is $G_{t+1}^{+E} = (V, (E_t \setminus D_t) \cup I_t)$. The system completes deletion repair before processing insertions. Figure 3 follows this order: 1–3 perform dependency invalidation and deletion repair (Section 5.1), followed by topology publication and insertion propagation in 4–6 (Section 5.2).

1 Dependency invalidation. When deletions invalidate a path supporting a vertex result, the GPU marks the vertices that depend on that path and resets their results for repair. The Grapin implementation records these marks in arrays indexed by vertex ID and scans all graph partitions to rebuild deletion work lists after each round.¹ Although the marks may cover only a small part of the graph, collecting them still examines unaffected vertices across the entire graph. We build a compact work list directly as vertices are marked.

The GPU first initializes an empty work list L_{del} and examines the deleted edges. For each deleted edge (u, v) , it marks v if its current result depends on the deleted edge. Only the thread that first marks v atomically reserves the next slot and appends its ID to L_{del} . The GPU then processes these initial vertices, scans their outgoing edges in G_t , and applies the same dependency check to each neighbor. Newly marked neighbors are appended to the same work list. Each subsequent round processes only the range appended in the preceding round, giving a breadth-first traversal of dependencies. When a round appends no new IDs, L_{del} contains all vertices marked for repair, which we denote by A . Its occupied slots are consecutive, each marked vertex appears once, and unaffected vertices occupy no slots. The GPU resets the stored and tentative results of these vertices to ∞ and transfers their IDs to the CPU. Reusing this work list directly avoids another graph-wide scan to collect A and passes only $|A|$ IDs to CPU preparation.

¹The all-vertex initialization and partition scheduling describe the local implementation examined in this work of [Efficient Graph Data Access for Out-of-Memory GPU Streaming Graph Processing] (PVLDB 2025, Section 4).

2 Constructing the predecessor view. A marked vertex may have an alternative path through an unaffected predecessor whose result has not changed. For example, after deleting (u, v) , a remaining edge (x, v) may restore v even though x has no update to trigger further processing. Repair therefore requires the remaining incoming edges of the vertices in A :

$$I(A) = \{(u, v) \in E(G_{t+1}^{-E}) \mid v \in A\}. \quad (5.1)$$

The GPU supplies only the destination IDs in A . The CPU obtains their predecessors from the reverse index maintained in Section 4.2. The reverse index describes the incoming edges of vertices in G_{t+1}^{-E} by combining base lists and signed delta records. For each $v \in A$, the CPU merges the two lists in source order and retains sources with positive remaining multiplicity. This constructs the current predecessor view without scanning the outgoing adjacency lists of all sources, confining predecessor discovery to the incoming records associated with A .

For sufficiently large A , CPU workers partition the destinations in A into disjoint groups and process these groups independently. Each worker first writes the merged predecessor lists to a private buffer for its group and records the number of retained predecessor IDs for each destination. A prefix sum over these list lengths determines destination offsets and assigns disjoint ranges in the final source array. Workers then copy their buffered predecessor IDs in parallel without synchronization on a shared append buffer. Buffering also avoids traversing the incoming base and delta records again after determining the list lengths: each destination's records are merged only once during construction.

The resulting view stores each destination's predecessor IDs contiguously as an incoming list, with offsets delimiting the lists in the order of A . The CPU transfers these offsets and source IDs to the GPU, where a repair task can directly access all predecessors of its assigned destination. Vertex results remain on the GPU. If M_A is the number of emitted predecessor entries, this representation occupies $O(|A| + M_A)$ space. Construction work is proportional to $|A|$ plus the base and delta records examined for these destinations. Thus, the affected set identified by GPU invalidation directly determines the predecessor data prepared by the CPU and transferred for repair.

3 Repairing vertex results. Using the prepared incoming edges, the GPU repeatedly improves the result of each $v \in A$ with candidates derived from its predecessors. For SSSP, this update is:

$$d(v) \leftarrow \min \left\{ d(v), \min_{(u,v) \in I(A)} [d(u) + w(u, v)] \right\}. \quad (5.2)$$

The iteration continues until no result changes, with an empty predecessor set contributing ∞ . A warp cooperatively scans each incoming list. Between rounds, the CPU checks whether the GPU recorded any result changes to determine whether repair has converged. Better results propagate through the incoming edges of A , updating affected vertices whenever a better valid candidate is found. After this process, the correct vertex results in G_{t+1}^{-E} are restored, and vertices with no remaining path remain at ∞ .

The local Grapin implementation propagates deletion invalidation first, then restores vertex results after applying both deletions

and insertions through a traversal initialized from all vertices.² By completing deletion repair over the incoming lists of A before insertion processing begins, our system restores the correct vertex results in G_{t+1}^{-E} at 3. Insertion repair can then start only from destinations whose results improve through inserted edges, avoiding the construction and initial processing of a work list covering all vertices.

5.2 Topology Publication and Insertion Propagation

4 Publishing the updated graph. Isolating updates to individual sources keeps descriptors for unchanged sources valid, enabling sparse publication. After deletion repair, the CPU applies insertions, updates the reverse index, and combines changed sources into $S = S^- \cup S^+$. Using Section 4.3's mechanism, it publishes final descriptors only for S and repairs their cached lists where possible. Once publication completes, the GPU can traverse the final graph without a full descriptor reload.

5 Building the initial work list. Even when inserted edges improve only a few destinations, collecting active vertices by scanning vertex marks across all partitions incurs work over unaffected vertices. As in 1, we construct a compact work list by appending vertex IDs when the corresponding state changes. After publication completes, GPU threads relax the inserted edges in parallel using atomic minimum updates. Each destination whose tentative result improves is appended at most once to the initial work list. Since 3 has already restored the results for the graph after deletion, these destinations suffice to initiate insertion repair. This avoids a graph-wide collection pass and confines initial adjacency traversal in 6 to vertices with improved results. Subsequent rounds use the same direct construction to keep the work localized throughout the iterations.

6 Propagating result improvements. For each vertex in the current work list, a GPU thread block checks whether its latest tentative result is better than its stored result. If so, it commits the improvement and examines the outgoing edges. We store adjacency chunks in pinned host memory mapped to the GPU. The block uses the published descriptor to read the vertex's contiguous adjacency list through zero-copy access. Whenever an edge improves its destination's tentative result, the destination is added to the next work list. An atomic minimum preserves the best concurrent update, and a per-vertex tag containing the batch and round admits each destination at most once per round. A later improvement may add it again in a subsequent round. Thus, successful result updates directly build the next work list, avoiding scans of unrelated vertices to reconstruct partition work lists.

Compact work lists reduce traversal work, but coordinating their rounds can still impose overhead when only a few vertices remain active. In the local Grapin implementation, the CPU dispatches partition processing, rebuilds work lists, and checks partition activity to determine convergence.³ We keep insertion propagation within

²The all-vertex initialization and partition scheduling describe the local implementation examined in this work of [Efficient Graph Data Access for Out-of-Memory GPU Streaming Graph Processing] (PVLDB 2025, Section 4).

³The all-vertex initialization and partition scheduling describe the local implementation examined in this work of [Efficient Graph Data Access for Out-of-Memory GPU Streaming Graph Processing] (PVLDB 2025, Section 4).

a single cooperative GPU kernel. The GPU maintains the current and next work lists and uses grid-wide barriers to complete pending writes before exchanging the lists. It terminates when the next work list is empty. This combines scheduling driven by successful relaxations with GPU control of propagation rounds. It eliminates CPU work list orchestration, intermediate convergence checks, and repeated propagation launches between insertion rounds. The CPU waits for completion at the end of propagation.

Let X_k be the vertices whose results improve and whose outgoing edges are examined in round k . The edge work is

$$W_{\text{expand}} = \sum_k \sum_{u \in X_k} \deg_{G_{t+1}}^+(u). \quad (5.3)$$

This excludes adjacency reads from vertices without improvements, reducing edge processing and requests to host memory. Repeated improvements can still cause repeated reads. Section 5.3 describes an optional distance ordering to reduce them.

Algorithm 1: CPU–GPU cooperative processing of an update batch

```

ProcessBatch(G_t, state, deletions, insertions):
  CPU: P <- GroupUpdatesBySource(deletions, insertions)
  // 1 Dependency invalidation: retain discovered IDs in Ldel
  GPU: Ldel <- InvalidateDependenciesAndAppendIDs(G_t, state,
    deletions)
    A == vertices in Ldel // reuse the work list directly
    ResetAffectedResults(A)
  GPU -> CPU: affected vertex IDs A

  // 2 Construct the predecessor view for A from the reverse index
  CPU: ApplyEffectiveDeletionsAndCommitReverse(P)
    incoming <- ParallelMaterializeIncoming(A)
  CPU -> GPU: incoming list offsets and source IDs
  // 3 Repair affected results
  GPU: RepairAffectedResultsUntilUnchanged(A, incoming)
    // CPU controls ordinary repair rounds

  // 4 Publish the updated graph
  CPU: ApplyInsertionsAndCommitReverse(P)
    S <- UnionOfChangedSources(P)
  CPU -> GPU: final descriptor patches for S
  GPU: PublishDescriptorsAndRepairCachedLists(S)
    WaitForPublicationBeforePropagation()
  // 5 Build the initial work list
  L <- RelaxInsertedEdgesAndAppendImprovements(insertions)
  // 6 Propagate result improvements
  within one cooperative kernel:
    while L is not empty:
      Lnext <- empty
      for each u in L in parallel:
        if CommitLatestImprovement(u):
          for each (u, v) in final outgoing adjacency:
            if AtomicRelax(u, v) succeeds:
              AppendOnce(v, Lnext)
      synchronize all producers
      swap(L, Lnext)

```

5.3 Adapting Cooperative Execution to Workload Cost

Selective processing limits which vertices participate in an update, but its remaining cost depends on the workload. Long propagations can repeatedly revisit the same adjacency lists, whereas large update

batches can make CPU preparation a bottleneck. We address these costs through ordered propagation and shared batch preparation, respectively.

Ordered propagation for long propagation chains. In this workload, an update affects vertices many hops away, and competing paths may deliver better results in later rounds. A vertex can therefore be reactivated several times before convergence. The ordinary deletion repair in Section 5.1 scans every prepared incoming list each round, including those of vertices whose results do not change. The insertion work lists in Section 5.2 exclude inactive vertices, but an active vertex can still propagate a result that a later improvement supersedes. Each reactivation may then cause another scan of its outgoing edges. Compact work lists alone cannot eliminate this repeated edge processing.

Our ordered mode couples the representation used for deletion repair with the order in which the GPU processes pending work. For deletion repair, the CPU converts edges internal to A from the prepared incoming view into a local outgoing CSR representation. The GPU initializes candidate results from unaffected boundary predecessors and places vertices with finite candidates in a pending work list. It selects the smallest nonempty distance bucket and expands only the selected vertices through the local outgoing edges. Successful relaxations schedule further work, while unselected vertices remain pending. This changes repair from repeated pulls over all incoming lists of A to expansion from selected pending vertices. At convergence, the GPU writes the repaired results back to the main state and reconstructs affected parents from tight incoming edges.

Insertion propagation applies ordering within the existing cooperative kernel and requires no additional CPU graph representation. The GPU finds the minimum pending tentative result m and processes vertices whose tentative results fall in $[m, m + \Delta - 1]$. It carries the remaining vertices into the next work list without committing their tentative results. Carried vertices and destinations improved by relaxation use the same deduplication tags, preserving pending work without duplicate entries in that round. For the current SSSP weights in $[1, 128]$, we use $\Delta = 128$. Deletion selects fixed buckets indexed by $\lfloor d/\Delta \rfloor$, whereas insertion uses a moving window. Both schedules allow later improvements to reactivate a vertex.

These changes reduce repeated adjacency scans by prioritizing better pending results before they propagate further. The benefit comes from less edge processing and does not require fewer propagation rounds. It must outweigh construction and transfer of the local CSR, initialization of the host local-ID map over all vertices, and GPU selection and carry operations. Ordered deletion also retains host reads of aggregate control counters between rounds, while insertion stays within one cooperative kernel. We therefore expose ordering as an optional mode. Its benefit for BFS must be assessed separately from weighted SSSP.

Shared preparation for large update batches. A large batch contains many update records and can modify the adjacency lists of many sources. Even when few vertex results ultimately change, the CPU must prepare topology updates, maintain the reverse index, and identify descriptors for publication. In the regular path, materializing a separate source-plan array and copying effective changes into reverse-index preparation add work proportional to the batch's maintenance data. Concatenating and sorting the changed-source

lists introduces another pass over information already ordered within each phase. These costs can dominate the selective GPU computation.

We reduce this duplication by sharing source plans and passing their outputs directly to subsequent maintenance steps. The CPU groups the batch once by source and exposes separate deletion and insertion views. Within each phase, the large-batch mode retains a single copy of each source plan and uses indices to share it among effective-change generation, allocation, adjacency updates, and retirement. It then transfers ownership of the effective-change buffer to reverse-index preparation, eliminating the input copy. The reverse index still performs the sorting and merging needed to construct its own representation. After both phases, an optional linear merge of their ordered changed-source lists produces the unique publication set without sorting their concatenation.

This organization reduces redundant plan materialization, buffer copying, and publication sorting as the batch grows. Source grouping also exposes independent adjacency lists to parallel CPU maintenance and supports the predecessor preparation in Section 5.1.

Sharing preparation preserves the phase contract: deletion repair converges before insertion processing, and final descriptors become visible before GPU propagation. The GPU retains vertex results throughout this process.

To keep repeated handoffs from offsetting these savings, the system reuses incoming-edge workspace, publication staging buffers, and completion events across batches. Valid cache contents repaired during publication can also satisfy Section 4.3’s condition for skipping eviction, compaction, and loading. Hotness scoring and candidate selection still execute. These mechanisms reduce preparation and maintenance costs. They do not improve cache hits in the insertion kernel, which reads mapped host adjacency.

6 Experimental Evaluation

7 Related Work

8 Conclusion